BRAINWARE UNIVERSITY

BRAINWARE UNIVERSITY

PYTHON PROJECT-BASED LEARNING REPORT (WEEK 4)

1. Basic Information

Title of the Project:	**Password Strength Checker & Email Validation System** (Program 1 & 2)
Name(s) of Student(s):	**Soumyadwip Das (BWU/BTS/25/169)** **Ankita Paul (BWU/BTS/25/180)** **Sonia Naskar (BWU/BTS/25/195)** **Joyeta Mondal (BWU/BTS/25/214)** **Sayantan Bharati (BWU/BTS/25/503)**
Programme & Semester:	**B.Tech CSE & 3rd Semester**
Course Name & Code:	**Python Programming Lab (BES09013)**
Name of the Guide/Supervisor:	**MR. PRANAB GHARAI & MR. INDRANIL DAS**

2. Overview

This week's laboratory work comprises two console-based Python programs that continue the rule-based, string-validation theme introduced earlier in the course. Program 1 develops a Password Strength Checker that scores password composition against character-class rules. Program 2 develops an Email Validation System that parses and checks an email address's structural correctness using only built-in string methods, without regular expressions. Both programs share a common boxed-menu terminal interface built from reusable box-drawing helper functions, keeping the two deliverables consistent in look and feel while remaining independent in their validation logic.

PROGRAM 1 — PASSWORD STRENGTH CHECKER

2.1 Introduction and Background

Weak passwords remain one of the most common causes of account compromise, which makes automated password-strength feedback a practical and widely used security feature. This week's laboratory work focuses on developing a console-based Password Strength Checker in Python that evaluates a candidate password against a set of composition rules and returns clear, actionable feedback to the user.

Following the account-security concepts touched on during Week 2 Program 2 (ATM Transaction Simulator), this program shifts focus from credential verification to credential quality assessment, exploring character-level string analysis, rule-based classification, and formatted terminal output using only Python's built-in string methods.

2.2 Objectives

- Design and implement a console-based password strength checker with a boxed menu interface
- Develop a validation routine that checks password length, digits, lowercase, uppercase and special characters
- Classify password strength into weak, medium, and strong categories based on the checks satisfied
- Implement a visual strength bar and a per-requirement checklist for user feedback
- Explore Python's built-in character-testing methods (isdigit, islower, isupper) for rule-based validation
- Test the checker against passwords of varying length and composition to verify correct classification

2.3 Problem Statement / Driving Question

How can a password's composition be analysed programmatically to give a user immediate, understandable feedback on its strength, and how can weak, medium, and strong passwords be reliably distinguished using simple rule-based checks?

2.4 Methodology

Program Structure: Reused the box-drawing helper functions (box_top, box_bottom, box_divider, box_line, print_boxed) to render a consistent bordered menu and result interface, and displayed an ASCII-art banner and centred title on program start-up.

Password Validation Logic: Developed password_validator() to test a password against five criteria — minimum length, digit, lowercase letter, uppercase letter, and special character — storing each requirement and its pass/fail result in a checks dictionary, then classifying the password as weak, medium, or strong based on which combinations of criteria were satisfied.

Result Presentation: Implemented print_result() to render a proportional strength bar reflecting the number of criteria met, an [OK]/[NO] checklist against each requirement, and word-wrapping so feedback fits neatly inside the bordered box.

Menu-Driven Control Flow: Built a continuous while-loop menu offering Check Password and Exit options, and tested the checker with short, letters-only, and fully mixed-character passwords to confirm correct classification.

2.5 Expected Outcomes / Deliverables

- A working console-based Password Strength Checker classifying passwords as weak, medium, or strong
- A visual strength bar and detailed requirement checklist for user feedback
- A clear, boxed terminal interface consistent with earlier laboratory programs
- Documented understanding of rule-based string validation using Python's built-in character methods
- Identified enhancement opportunities such as dictionary/common-password checks and entropy-based scoring for future work

2.6 Core Logic

The password_validator() function is the heart of the program. It runs five independent character-level checks against the input password and combines their results to decide a strength category. This rule-based approach keeps the logic transparent and easy to extend with additional checks in future iterations.

```
def password_validator(password):
    has_num = any(ch.isdigit() for ch in password)
    has_lower = any(ch.islower() for ch in password)
    has_upper = any(ch.isupper() for ch in password)
    has_special = any(ch in SPECIAL_CHARS for ch in password)
    long_enough = len(password) >= 8

    if not long_enough:
        return "weak", "Password needs to have at least 8 characters.", checks
    if has_num and has_lower and has_upper and has_special:
        return "strong", "Password is Strong! Good to go.", checks   # all criteria met
    elif has_num and has_lower:
        return "medium", "Try adding an uppercase letter and a special character.",
checks
```

Note: the strength categories currently rely on fixed rule combinations rather than a weighted score; introducing an entropy- or scoring-based model is planned as a future enhancement.

2.7 Output Screenshot

PROGRAM 2 — EMAIL VALIDATION SYSTEM

3.1 Introduction and Background

Validating user-submitted data is a fundamental requirement in almost every software system, and email addresses are among the most frequently validated fields during account registration and communication workflows. This week's laboratory work focuses on developing a console-based Email Validation System in Python that checks a candidate email address against a series of structural and formatting rules and reports a clear, specific reason whenever validation fails.

Continuing from the rule-based validation approach explored in Program 1 (Password Strength Checker), this program applies similar character-level analysis to a different domain: parsing an email address into its local part and domain, and verifying each part independently using Python's built-in string methods, without relying on regular expressions or any external validation library.

3.2 Objectives

- Design and implement a console-based email validation system with a boxed menu interface
- Develop a validation routine that checks for the presence of a single '@' symbol and absence of spaces
- Implement separate checks for the local part (before '@') and the domain part (after '@')
- Validate that the domain contains exactly one dot and a properly formed top-level domain (TLD)
- Restrict accepted TLDs to a defined set of common domains (com, org, edu, net, gov, mil)
- Test the validator against valid, malformed, and edge-case email addresses to verify correct behaviour

3.3 Problem Statement / Driving Question

How can a submitted email address be checked programmatically for structural validity — correct placement of '@', a well-formed local part and domain, and an accepted top-level domain — and how can specific, informative error messages be returned for each type of formatting mistake?

3.4 Methodology

Program Structure: Reused the box-drawing helper functions (box_top, box_bottom, box_divider, box_line, print_boxed) to render a consistent bordered menu and result interface, with an ASCII-art banner and centred title on start-up.

Local Part and Domain Validation: Developed name_check() to confirm the local part of the email contains only alphanumeric characters and is non-empty, and domain_check() to confirm the domain contains exactly one dot, with non-empty alphanumeric segments on each side and a TLD of at least two characters.

Overall Email Validation Logic: Implemented email_checker() to run preliminary checks first — presence of '@', absence of spaces, minimum length, and a single '@' occurrence — then split the email into local and domain parts and validated each using name_check() and domain_check(), restricting the accepted TLD to a fixed set (com, org, edu, net, gov, mil) and returning a descriptive error naming the invalid TLD when the check fails.

Menu-Driven Control Flow: Built a continuous while-loop menu offering Check Email and Exit options, and tested the validator with missing '@', multiple '@' symbols, spaces, malformed domains, and unsupported TLDs to confirm each returns the correct, specific error message.

3.5 Expected Outcomes / Deliverables

- A working console-based Email Validation System that checks structural correctness of an email address
- Specific, rule-by-rule error messages that explain exactly why an email failed validation
- A clear, boxed terminal interface consistent with earlier laboratory programs
- Documented understanding of rule-based string parsing and validation using Python's built-in string methods
- Identified enhancement opportunities such as regex-based validation and an expandable/configurable TLD list for future work

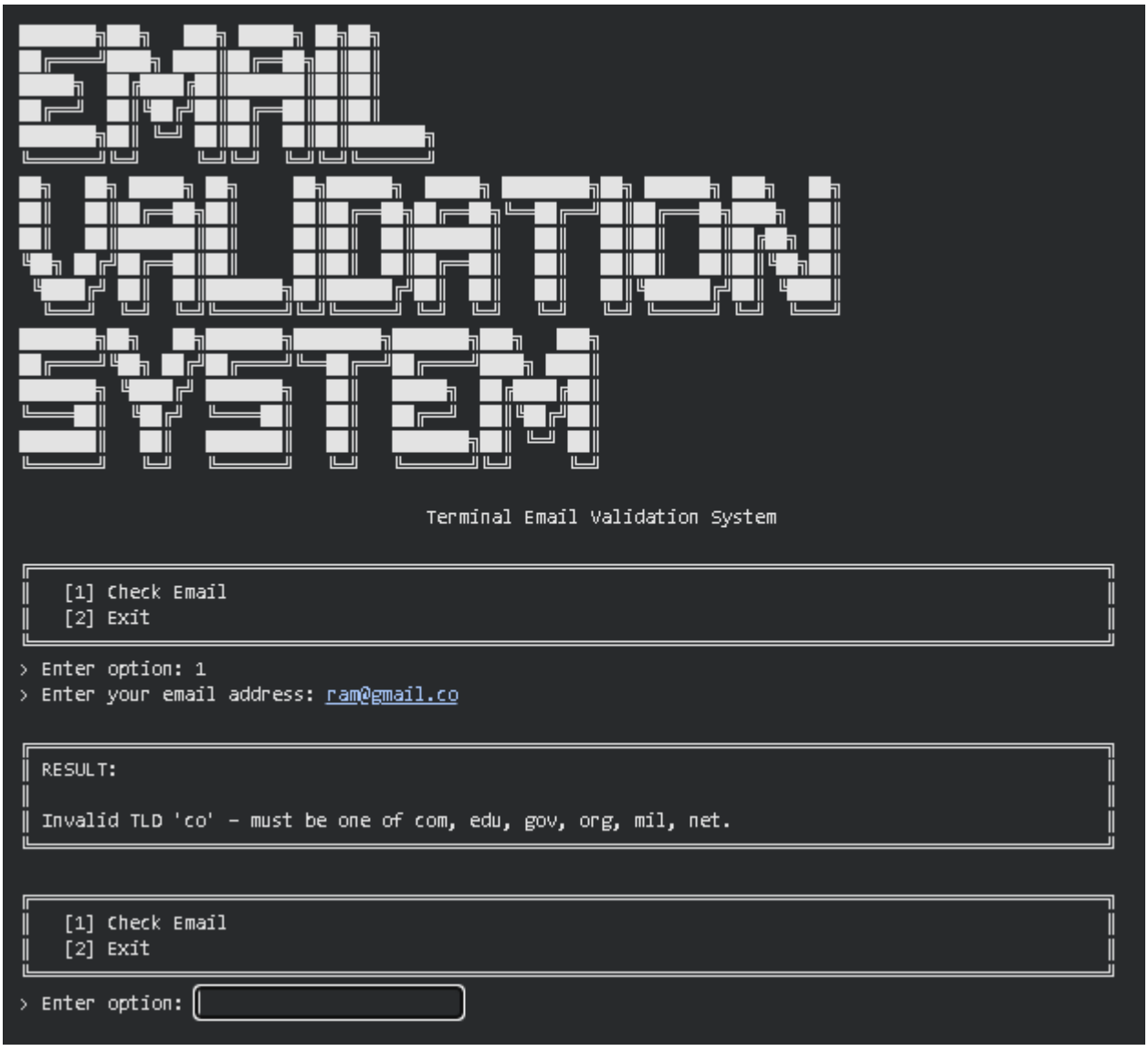
3.6 Core Logic

The domain_check() function is central to the validator's accuracy. It parses the domain portion of the email, confirms it contains exactly one dot, and verifies that both the name and TLD segments are non-empty, alphanumeric, and that the TLD meets a minimum length. This isolates domain-specific validation from the local-part and overall '@'-placement checks, keeping each rule easy to test and extend.

```
def domain_check(domain):
    # Must have exactly one dot, and only letters/digits before and after
    if domain.count('.') != 1:
        return False
    local, tld = domain.split('.')
    if not local or not tld:
        return False
    if not local.isalnum() or not tld.isalnum():
        return False  # reject symbols/spaces in domain parts
    if len(tld) < 2:
        return False  # TLD must be at least 2 characters
    return True
```

Note: the system currently restricts valid emails to a fixed TLD set and alphanumeric-only local/domain parts, so addresses using hyphens, subdomains, or newer TLDs are rejected; broadening this coverage is planned as a future enhancement.

3.7 Output Screenshot



4. Project Timeline

Week	Date	Activity	Status
Week 1	10/07/2026	Student Grade Management System	**Completed**
Week 2	17/07/2026	Program 1: Electricity Bill Calculator	**Completed**
Week 2	17/07/2026	Program 2: ATM Transaction Simulator	**Completed**
Week 3	24/07/2026	Program 1: Bank Management System	**Completed**
Week 3	24/07/2026	Program 2: Hotel Booking System	**Completed**
Week 3	24/07/2026	Program 3: Library Management System	**Completed**
Week 4	**31/07/2026**	**Program 1: Password Strength Checker**	**Current Week**
Week 4	**31/07/2026**	**Program 2: Email Validation System**	**Current Week**
Week 5	TBD	Text Encryption & Decryption Tool	**Upcoming**

5. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Core programming language used to implement both the password strength checker and the email validation system
str methods (isdigit, islower, isupper, isalnum, split, count)	Character-level checks and string parsing used to evaluate password composition and to validate the email local part, domain, and TLD
Python Dictionaries	Storing and iterating over named password requirement checks (Program 1)
Python Sets	Storing the collection of accepted top-level domains for fast membership checks (Program 2)
String Formatting (f-strings)	Rendering the strength bar, requirement checklist, and rule-specific validation error messages
Functions & Control Flow	Modularising validation logic (password rules; local-part, domain, and TLD checks) and boxed result rendering for both programs

6. References

- Python Official Documentation: https://docs.python.org/3/
- Python String Methods Reference: https://docs.python.org/3/library/stdtypes.html#string-methods
- OWASP Password Strength Guidance: https://owasp.org/www-community/controls/Password_strength
- IANA List of Top-Level Domains: https://www.iana.org/domains/root/db
- Brainware University - Machine Learning Course Guidelines

7. Signatures

Signature of Student(s): **Date of Submission:**

Signature of Guide/Supervisor: